

Padded-FAA Queue: progettazione e valutazione sperimentale in MoStream

Documentazione tecnica per attività di tesi

Progetto MoStream - 22 luglio 2026

Questa relazione descrive PaddedFAAQueue, una coda bounded Multiple Producer, Multiple Consumer progettata combinando allocazione dei ticket mediante Fetch-And-Add, sequence number per slot, indicizzazione power-of-two e isolamento delle celle su cache line. L'obiettivo è ridurre sia le collisioni CAS sui contatori globali sia il false sharing fra slot adiacenti. Nella campagna sperimentale eseguita su 18 configurazioni Padded-FAA ha ottenuto il miglior throughput in 13 casi, risultando la migliore scelta generale fra le implementazioni analizzate. La coda conserva tuttavia ticket irrevocabili, head-of-line blocking e un consumo di memoria superiore alle varianti compatte.

Contesto e obiettivo

MoStream utilizza code bounded MPMC per trasferire messaggi fra gli stage delle pipeline e, nel runtime cooperativo, per gestire ready queue e wait queue. La coda originale usa il ring buffer a sequence number attribuito a Dmitry Vyukov e prenota le posizioni mediante Compare-And-Swap. Gli esperimenti precedenti hanno mostrato che Fetch-And-Add scala meglio sotto contesa, mentre la coda di Erik Rigtorp beneficia dell'isolamento degli slot su cache line.

Padded-FAA nasce dall'unione delle caratteristiche risultate più promettenti:

- ticket FAA per push e pop bloccanti;
- contatori enqueue e dequeue separati su cache line;
- slot padded e allocati con allineamento di 64 byte;
- sequence number per generazione, con sincronizzazione ACQUIRE/RELEASE;
- capacità power-of-two e indice calcolato con una mask;
- CAS per le operazioni non bloccanti, nelle quali il ticket deve poter essere abbandonato;
- spin limitato e yield periodico durante le attese lunghe.

Struttura dati

La coda è un ring buffer di capacità fissa. La capacità deve essere una potenza di due e almeno pari a due. I campi principali sono:

slots	array di PaddedFAASlot[T]
size	capacità del ring
mask	size - 1
enqueue_pos	prossimo ticket producer
dequeue_pos	prossimo ticket consumer

Ogni slot contiene:

sequence	stato e generazione atomica della cella
data	payload Optional[T]
padding	byte necessari a raggiungere un multiplo di 64 byte

L'array è richiesto all'allocatore Mojo con alignment=64. In questo modo il primo slot inizia su una cache line e, poiché la dimensione di ogni slot è un multiplo di 64 byte, anche tutti gli slot successivi risultano isolati.

Protocollo dei sequence number

Per un producer con ticket p:

- sequence uguale a p: la cella è disponibile;
- sequence uguale a p+1: il messaggio è stato pubblicato.

Per un consumer con ticket c:

- sequence uguale a c+1: il messaggio è consumabile;
- dopo il consumo sequence viene impostato a c+size, preparando la cella per il giro successivo del ring.

Il sequence number evita l'ambiguità tra queue piena e vuota e distingue le diverse generazioni che attraversano la stessa posizione fisica.

Operazione push bloccante

Il producer ottiene una posizione unica con un singolo fetch_add RELAXED:

```
ticket = enqueue_pos.fetch_add(1)
slot = slots[ticket & mask]
attendi finché slot.sequence == ticket
scrivi il payload
slot.sequence.store(ticket + 1, RELEASE)
```

FAA impedisce che più producer competano ripetutamente sullo stesso valore del contatore, eliminando i retry CAS osservati nella MPMC originale. Il ticket è irrevocabile: una volta incrementato enqueue_pos, il producer deve completare la pubblicazione.

Operazione pop bloccante

Il consumer segue il protocollo simmetrico:

```
ticket = dequeue_pos.fetch_add(1)
slot = slots[ticket & mask]
attendi finché slot.sequence == ticket + 1
estrai il payload
slot.sequence.store(ticket + size, RELEASE)
```

La lettura ACQUIRE del sequence osserva il payload scritto prima della store RELEASE del producer. La store RELEASE del consumer rende visibile la liberazione dello slot al producer della generazione seguente.

Operazioni try_push e try_pop

Le operazioni non bloccanti non possono usare direttamente FAA. Se una operazione ottenesse un ticket e scoprisse di non poter completare, non potrebbe annullare in sicurezza l'incremento senza creare un buco permanente.

Padded-FAA usa quindi:

1. load della posizione corrente;
2. verifica della sequence prevista;
3. compare_exchange per riservare la posizione;
4. ritorno immediato in caso di cella non pronta o collisione CAS.

Il vantaggio FAA riguarda dunque principalmente push e pop bloccanti.

Padding e false sharing

Nelle code compatte più sequence number possono risiedere nella stessa cache

line. Producer e consumer che lavorano su slot distinti invalidano comunque la linea condivisa, generando traffico di coerenza non necessario. Padded-FAA assegna almeno una cache line a ogni cella.

Il beneficio cresce quando molti thread accedono contemporaneamente a slot adiacenti. Il costo è un aumento della memoria. Con slot minimi da 64 byte:

```
tab( ), box;
Capacità  Memoria slot minima  Note
16  1 KiB  trascurabile
1024  64 KiB  tipicamente compatibile con le cache
65536  4 MiB  possibile pressione su cache e TLB
```

Payload superiori a una cache line portano la dimensione dello slot al successivo multiplo di 64 byte.

Indicizzazione e memory ordering

La capacità power-of-two permette di calcolare l'indice con ticket & mask. Questo evita modulo e divisione usati nella coda Rigtorp per supportare capacità arbitrarie. I contatori di ticket usano Ordering.RELAXED perché l'ordine dei dati è stabilito dai sequence number ACQUIRE/RELEASE. enqueue_pos e dequeue_pos sono inoltre collocati in oggetti padded separati.

Questa scelta deve essere mantenuta con attenzione: rilassare le operazioni sul sequence number invaliderebbe il protocollo di pubblicazione del payload.

Politica di attesa

Durante l'attesa dello slot il thread esegue spin. Dopo 1024 osservazioni senza successo viene invocato sleep(0.0), cedendo occasionalmente la CPU allo scheduler. La politica cerca un compromesso fra bassa latenza e consumo CPU.

Rigtorp usa puro spin, che può aumentare il throughput su core dedicati ma risulta più aggressivo in oversubscription. La soglia di 1024 deve essere considerata un parametro sperimentale, non un valore universalmente ottimale.

API Mojo

La coda è definita in MoStream/Padded_FAA_queue.mojo e richiede un payload Copyable. Esempio:

```
from MoStream.Padded_FAA_queue import PaddedFAAQueue

var queue = PaddedFAAQueue[Int](1024)
queue.push(42)
var value = queue.pop()
```

Metodi disponibili:

- push(item): inserimento bloccante;
- pop(): estrazione bloccante;
- try_push(item): restituisce il payload se non inserito;
- try_pop(): restituisce Optional[T];
- estimated_len(): snapshot approssimativo della lunghezza.

Metodologia sperimentale

Ogni producer inserisce un intervallo disgiunto di interi. L'ultimo producer pubblica un sentinel per consumer. I consumer accumulano conteggio e checksum in variabili locali, evitando un contatore atomico condiviso nel percorso temporizzato. Dopo il join vengono verificati:

conteggio = numero totale di messaggi
checksum = $N \times (N - 1) / 2$

Sono state provate le topologie 1P-1C, 2P-2C, 4P-4C, 8P-8C, 1P-8C e 8P-1C; le capacità 16, 1024 e 65536; 250000 messaggi per producer; 10 ripetizioni per configurazione. Tutte le esecuzioni Padded-FAA hanno superato il controllo di correttezza.

Ambiente: Intel Xeon Gold 5512U, 28 core e 56 thread hardware, x86-64, un nodo NUMA, Linux 5.15.0-168-generic e Mojo 1.0.0b1.

Risultati principali

La tabella riporta milioni di messaggi al secondo. Best indica il vincitore della configurazione fra CAS, FAA, Hybrid, Rigtorp e Padded-FAA.

```
tab( ), box;
Topologia Capacità FAA Rigtorp Padded-FAA Best
1P-1C 16 5.47 3.74 3.87 FAA
1P-1C 1024 5.97 3.73 4.16 FAA
1P-1C 65536 6.24 2.75 15.54 Padded-FAA
2P-2C 16 6.84 7.49 8.21 Padded-FAA
2P-2C 1024 8.02 9.36 10.01 Padded-FAA
2P-2C 65536 11.22 11.89 12.11 Padded-FAA
4P-4C 16 8.34 8.24 10.61 Padded-FAA
4P-4C 1024 8.81 9.41 11.86 Padded-FAA
4P-4C 65536 10.81 10.16 11.92 Padded-FAA
8P-8C 16 9.50 8.40 9.56 Padded-FAA
8P-8C 1024 8.65 8.45 10.42 Padded-FAA
8P-8C 65536 9.39 9.69 10.35 Padded-FAA
1P-8C 16 4.60 7.91 7.62 Rigtorp
1P-8C 1024 4.51 7.78 7.98 Padded-FAA
1P-8C 65536 4.31 6.62 6.72 Hybrid-2
8P-1C 16 5.56 7.94 9.30 Padded-FAA
8P-1C 1024 5.37 7.34 9.44 Padded-FAA
8P-1C 65536 5.77 7.53 9.37 Hybrid-1
```

Padded-FAA è risultata prima in 13 configurazioni su 18. Ha vinto tutte le configurazioni bilanciate da 2P-2C a 8P-8C e due delle tre configurazioni 8P-1C. Il vantaggio su FAA e Rigtorp è particolarmente evidente in 4P-4C con capacità 1024, dove raggiunge 11.86 Mmsg/s contro 8.81 e 9.41.

Interpretazione dei risultati

Workload bilanciati

La combinazione di ticket distinti e slot isolati riduce due fonti diverse di contesa: collisioni sul contatore e invalidazioni fra sequence adiacenti. Padded-FAA domina quindi le topologie bilanciate con almeno due producer e due consumer.

Workload sbilanciati

Padded-FAA è molto competitiva ma non sempre prima in 1P-8C. Una coda SPMC specializzata potrebbe rimuovere completamente l'RMW atomico dal producer e mantenere FAA soltanto sul lato consumer. Analogamente una MPSC specializzata può semplificare il lato consumer.

1P-1C

Con capacità piccole FAA compatta evita il footprint degli slot padded e risulta superiore. Per questo Padded-FAA non sostituisce una vera SPSC, che può evitare del tutto FAA e CAS sui contatori posseduti da un solo thread.

Capacità elevata

Il risultato 1P-1C con capacità 65536 è molto elevato e deve essere replicato su macchina isolata prima di formulare conclusioni causali. Più in generale, cache residency, prefetching e TLB possono cambiare l'effetto del padding.

Confronto qualitativo

```
tab( ), box;
Proprietà CAS-MPMC FAA Rigtorp Padded-FAA
Ticket bloccanti CAS FAA FAA FAA
Slot padded No No Sì Sì
Indice mask mask modulo mask
Ticket ordering RELAXED RELAXED SEQ_CST RELAXED
Attesa spin spin/yield spin spin/yield
Capacità arbitraria No No Sì No
Memoria per slot compatta compatta >=64 B >=64 B
```

Garanzie e limiti

Padded-FAA non è wait-free. Dopo `fetch_add` un thread possiede un ticket che non può essere annullato. Se il thread viene sospeso prima di pubblicare o liberare lo slot, le operazioni successive possono subire head-of-line blocking.

Altri limiti:

- maggiore uso di memoria rispetto alla FAA compatta;
- possibile pressione su cache e TLB con capacità grandi;
- `try_push` e `try_pop` non beneficiano del percorso FAA;
- nessuna prova formale di linearizzabilità inclusa nello studio;
- nessun test mirato con thread sospeso dopo la prenotazione;
- misurato throughput, non latenza p95/p99, energia o fairness;
- risultati ottenuti su una sola macchina e una sola versione di Mojo.

Integrazione consigliata in MoStream

Padded-FAA è la migliore candidata come coda generale per communicator con push/pop bloccanti e contesa medio-alta. L'integrazione dovrebbe essere inizialmente opzionale, mantenendo FAA compatta e CAS come baseline.

Per ottenere il massimo senza un branch per messaggio, la selezione deve avvenire durante la costruzione del tipo concreto di Communicator. Un wrapper runtime `AdaptiveMPMCQueue` è stato prototipato, ma il dispatch nel percorso caldo ha perso circa il 28 percento rispetto alla Padded-FAA diretta nel test 4P-4C.

Strategia proposta:

1. Padded-FAA come default MPMC per topologie con almeno due thread concorrenti;
2. SPSC dedicata per 1P-1C;
3. SPMC padded specializzata per 1P-NC;
4. MPSC padded specializzata per NP-1C;
5. CAS o Hybrid dove le garanzie di progresso sono più importanti del throughput.

Lavoro futuro

1. Implementare `PaddedSPMCQueue` eliminando l'atomica dal producer singolo.
2. Implementare `PaddedMPSCQueue` eliminando l'atomica dal consumer singolo.
3. Confrontare slot da 64, 128 byte e layout compatto selezionabile.
4. Misurare cache miss, invalidazioni, context switch e consumo CPU con perf.
5. Misurare p50, p95, p99 e latenza massima.

6. Eseguire stress test con sospensione del proprietario di un ticket.
7. Ripetere con almeno 30 campioni, core isolati e pinning controllato.
8. Valutare payload di dimensioni differenti e pipeline MoStream end-to-end.

Conclusione

Padded-FAA è la migliore implementazione generale fra quelle analizzate in termini di throughput. La combinazione di FAA, slot allineati, sequence number, mask e memory ordering rilassato ha ottenuto il primo posto in 13 scenari su 18. Il risultato mostra che il false sharing degli slot era una componente rilevante oltre alla contesa sui contatori globali.

La coda non è universalmente dominante: FAA compatta resta preferibile in alcuni casi 1P-1C, mentre Rigtorp e Hybrid possono vincere specifici workload SPMC. Inoltre padding, ticket irrevocabili e head-of-line blocking introducono compromessi di memoria e progresso. La formulazione corretta per la tesi è che Padded-FAA rappresenta il miglior compromesso di throughput osservato e la baseline più promettente per successive specializzazioni SPSC, SPMC e MPSC.

File riproducibili

- MoStream/Padded_FAA_queue.mojo: implementazione;
- Tests/test_padded_faa_queue.mojo: test funzionale;
- Benchmarks/QueueBenchmark/queue_benchmark.mojo: benchmark concorrente;
- Benchmarks/QueueBenchmark/run_suite.py: matrice e statistiche;
- Benchmarks/QueueBenchmark/results_padded_faa.csv: risultati completi.

Riferimenti

- Dmitry Vyukov, Bounded MPMC queue, 1024cores.net.
- Erik Rigtorp, MPMCQueue, <https://github.com/rigtorp/MPMCQueue>.
- Mojo standard library, alloc con allineamento esplicito, https://docs.modular.com/mojo/std/memory/unsafe_pointer/alloc/.